Git: ramificación, integración y colaboración

Módulo PI2 · ASIR2

Paco Aldarias

30 de octubre de 2026

Índice

1. Antes de empezar **1**

2. Estrategias de ramificación **2**
2.1. GitHub Flow: opción recomendada para el proyecto 2
2.2. Otras estrategias 2

3. Estrategias de fusión **2**

4. Pull request **3**
4.1. Flujo de trabajo 3
4.2. Lista de revisión para proyectos ASIR 3

5. Conflictos **3**
5.1. Resolver un conflicto durante un merge 3

6. Reset y recuperación **4**

7. Stash (guardar cambios temporalmente) **4**

8. Consejos de trabajo **5**

9. GitHub Education y Copilot para estudiantes **5**

Guía práctica para trabajar en equipo con Git y GitHub en proyectos de administración de sistemas: documentación, scripts, configuraciones y automatización.

> **Objetivo.** Al terminar, podrás crear ramas de trabajo, integrar cambios, revisar propuestas, resolver conflictos y recuperar cambios locales con seguridad.

1. Antes de empezar

Git guarda el historial de un proyecto mediante *commits*. Cada commit es una versión identificable; una rama permite desarrollar una línea de trabajo sin modificar directamente otra. GitHub aloja repositorios y añade herramientas de colaboración como *pull requests* (PR).

Comprueba que tienes Git instalado y configura la identidad que aparecerá en tus commits:

```
git --version
git config --global user.name "Tu nombre"
git config --global user.email "tu-correo@example.com"
```

La identidad de Git no configura por sí sola el acceso a GitHub. Para descargar o publicar repositorios privados, autentícate con el método recomendado por GitHub (por ejemplo, Git Credential Manager o SSH). No incluyas contraseñas ni tokens en la URL del repositorio.

2. Estrategias de ramificación

Una rama debe representar un cambio concreto y tener una vida razonablemente corta. Así es más fácil revisar, probar e integrar el trabajo.

2.1. GitHub Flow: opción recomendada para el proyecto

1. Mantén `main` como rama estable.
2. Actualiza `main` antes de empezar.
3. Crea una rama descriptiva para una tarea.
4. Realiza commits pequeños y publica la rama.
5. Abre una PR, revisa y prueba los cambios.
6. Integra la PR y elimina la rama terminada.

```
git switch main
git pull --ff-only
git switch -c docs/diagrama-red
# Edita y prueba los archivos
git add docs/diagrama-red.md
git commit -m "docs: documentar el diagrama de red"
git push -u origin docs/diagrama-red
```

Nombres útiles de rama:

- `feat/monitorizacion`: nueva funcionalidad.
- `fix/dns-resolucion`: corrección.
- `docs/guia-backup`: documentación.
- `chore/actualizar-dependencias`: mantenimiento.

2.2. Otras estrategias

- **Trunk-based development:** el equipo integra cambios pequeños con mucha frecuencia en una rama principal. Requiere pruebas automáticas y coordinación para mantenerla estable.
- **Git Flow:** organiza el trabajo con ramas de larga duración como `develop` y ramas temporales `feature/`, `release/` y `hotfix/`. Puede ser útil con versiones planificadas, pero añade complejidad y no suele ser necesario para un proyecto pequeño de clase.

Criterio práctico: para un equipo ASIR pequeño, usa `main` más ramas cortas por tarea y PR. Protege `main` para evitar cambios directos sin revisar.

3. Estrategias de fusión

Fusionar incorpora los cambios de una rama en otra. Actualiza primero la rama de destino y comprueba el estado del repositorio:

```
git switch main
git pull --ff-only
git merge --no-ff docs/diagrama-red
```

Elige la estrategia según el historial que necesite el equipo:

- **Merge commit:** conserva los commits originales y registra explícitamente la integración. Es útil cuando interesa ver el trabajo de la rama como una unidad.
- **Squash and merge:** combina los cambios de la rama en un solo commit en `main`. Mantiene la rama principal más compacta; los commits intermedios no aparecen individualmente en ella.
- **Rebase:** vuelve a aplicar los commits sobre la punta más reciente de la rama destino y deja un historial lineal. Evítalo en ramas compartidas o publicadas si obliga a reescribir commits que otras personas ya usan.

Acuerda una estrategia para el repositorio y aplícala de forma consistente. Antes de fusionar, ejecuta las comprobaciones pertinentes: por ejemplo, validar la sintaxis de scripts, revisar archivos de configuración y probar el despliegue en un entorno seguro.

4. Pull request

Una **pull request** propone integrar cambios y permite revisarlos antes de fusionarlos. No es solo un botón para mezclar ramas: documenta la intención, facilita la revisión y muestra las comprobaciones automáticas.

4.1. Flujo de trabajo

1. Publica tu rama con `git push -u origin nombre-de-rama`.
2. En GitHub, crea una PR desde esa rama hacia `main`.
3. Explica qué problema resuelve, qué has cambiado y cómo lo has probado.
4. Solicita revisión a tus compañeros.
5. Atiende los comentarios y publica nuevos commits en la misma rama.
6. Espera a que las revisiones y comprobaciones requeridas terminen.
7. Fusiona con la estrategia acordada y elimina la rama si ya no se necesita.

4.2. Lista de revisión para proyectos ASIR

- ¿La descripción permite entender el objetivo y el alcance del cambio?
- ¿Se han probado los scripts y las configuraciones en un entorno adecuado?
- ¿Hay credenciales, claves privadas, tokens, datos personales o direcciones sensibles? No deben publicarse; si se expusieron, revócalos y sustitúyelos.
- ¿Los cambios en firewall, DNS, DHCP, permisos, copias de seguridad o servicios están explicados y son deliberados?
- ¿La documentación coincide con la configuración que se entrega?

5. Conflictos

Un conflicto aparece cuando Git no puede combinar automáticamente cambios incompatibles, a menudo porque dos ramas modificaron las mismas líneas. No significa que el repositorio esté perdido: hay que decidir qué contenido debe quedar.

5.1. Resolver un conflicto durante un merge

```
git status
# Abre los archivos marcados como "both modified"
```

En el archivo encontrarás marcadores parecidos a estos:

```
<<<<<<< HEAD
nameserver 192.0.2.10
=======
nameserver 192.0.2.53
>>>>>>> cambio-dns
```

Edita el archivo para dejar la configuración correcta, elimina todos los marcadores y comprueba el resultado. No elijas una versión automáticamente sin entender el efecto, especialmente en archivos de red y seguridad.

```
git diff
git add ruta/al/archivo
git status
git commit
```

`git status` indica los pasos restantes y puede preparar el mensaje del commit de merge. Si decides cancelar la fusión antes de completarla:

```
git merge --abort
```

Si el conflicto ocurre durante un rebase, resuelve los archivos, márcalos con `git add` y continúa con `git rebase --continue`. Para cancelar ese rebase, utiliza `git rebase --abort`. Un rebase puede cambiar identificadores de commits: coordínalo antes de actualizar una rama compartida.

6. Reset y recuperación

`git reset` mueve la referencia de la rama actual a otro commit. Según la opción elegida también modifica el área de preparación o los archivos de trabajo. Antes de recuperar o descartar nada, revisa:

```
git status
git diff
git diff --staged
git log --oneline --decorate -5
```

Opción	Mueve la rama	Conserva cambios preparados	Conserva cambios en archivos
`--soft`	Sí	Sí	Sí
`--mixed` (predeterminada)	Sí	No, los deja sin preparar	Sí
`--hard`	Sí	No	No, sobrescribe los archivos afectados

Ejemplos:

```
git reset --soft HEAD~1    # Deshacer el último commit y conservarlo preparado
git reset HEAD~1           # Quitar el último commit; conservar cambios sin preparar
```

Precaución: `git reset --hard` descarta cambios locales y puede hacer perder trabajo. No lo ejecutes sin revisar el estado y confirmar que puedes descartar esos cambios. Evítalo en commits ya compartidos: el resto del equipo puede seguir basándose en el historial anterior.

Si lo que quieres es restaurar un archivo concreto, identifica primero qué cambios vas a descartar:

```
git diff -- ruta/al/archivo
git restore -- ruta/al/archivo
```

Para quitar un archivo del área de preparación sin borrar sus cambios:

```
git restore --staged ruta/al/archivo
```

En repositorios compartidos, prefiere `git revert <commit>` para deshacer un commit publicado: crea un nuevo commit inverso y no reescribe el historial.

7. Stash (guardar cambios temporalmente)

`git stash` guarda temporalmente cambios locales sin crear un commit. Sirve, por ejemplo, si necesitas cambiar de rama antes de terminar una tarea. No es un sistema de copias de seguridad: consulta y aplica el contenido con cuidado.

```
git stash push -m "configuración pendiente de probar"
git stash list
git stash show -p stash@{0}
git stash apply stash@{0}
```

`apply` conserva la entrada en la lista. Cuando confirmes que los cambios se han recuperado correctamente, puedes eliminarla:

```
git stash drop stash@{0}
```

También puedes usar `git stash pop` para aplicar y retirar la entrada de la lista en una sola operación. Si hay conflictos, resuélvelos antes de continuar. Por defecto, los archivos sin seguimiento no se guardan; inclúyelos explícitamente si hace falta:

```
git stash push -u -m "incluye archivos nuevos"
```

8. Consejos de trabajo

- Comprueba `git status` antes y después de cada operación importante.
- Haz commits pequeños, coherentes y con mensajes que expliquen el cambio: `fix: validar el registro DNS` o `docs: añadir procedimiento de backup`.
- No incluyas contraseñas, tokens, claves SSH privadas, certificados privados, archivos `.env` con secretos ni copias de datos sensibles. Añade los patrones apropiados a `.gitignore`; ignorar un archivo después de haberlo versionado no elimina su historial.
- Si un secreto llegó al repositorio remoto, revócalo inmediatamente. Borrar el archivo en un commit posterior no invalida el secreto ni lo quita del historial.
- Usa `git diff` para revisar lo que vas a publicar y `git diff --staged` para comprobar exactamente lo preparado.
- Descarga cambios remotos antes de iniciar la integración. No uses `git push --force` en una rama compartida; si una reescritura es imprescindible, coordínala y usa `--force-with-lease` entendiendo su efecto.
- Prueba scripts con datos de prueba y en máquinas o redes de laboratorio. Revisa con especial cuidado comandos que cambien rutas, permisos, reglas de firewall, servicios o particiones.
- Documenta cómo reproducir el despliegue: requisitos, comandos, variables y comprobaciones. No documentes valores secretos.

9. GitHub Education y Copilot para estudiantes

Tener una cuenta gratuita de GitHub permite colaborar en repositorios públicos y privados. **GitHub Education** es un beneficio adicional: el alumnado que cumple los requisitos puede solicitar la verificación de estudiante y acceder a recursos educativos y ofertas del Student Developer Pack.

1. Crea o utiliza una cuenta personal en GitHub.
2. Consulta GitHub Education para estudiantes y solicita la verificación desde el portal.
3. Sigue las instrucciones actuales para acreditar que eres estudiante. La disponibilidad y las pruebas admitidas dependen de los criterios de GitHub; tener correo institucional, por sí solo, no garantiza la aprobación.
4. Cuando la cuenta esté verificada, revisa los beneficios disponibles en el Student Developer Pack y en tu perfil de GitHub Education.
5. GitHub indica que los estudiantes verificados pueden acceder gratuitamente a GitHub Copilot. Activa el beneficio con la cuenta verificada y comprueba en la configuración de Copilot qué plan, funciones y límites están disponibles: pueden cambiar.

En VS Code, inicia sesión con la cuenta que tiene el beneficio y configura la extensión oficial de GitHub Copilot. Puedes pedir explicaciones de comandos, ayuda para entender un script o sugerencias de documentación. Copilot puede equivocarse: revisa y prueba sus respuestas, especialmente antes de ejecutar comandos de administración o modificar sistemas. No pegues en el chat claves, contraseñas, datos personales ni información confidencial del centro o de una empresa.

Las actividades prácticas están en el documento independiente ud5-práctica.md y su versión PDF.